

Milestone Four

Primary Objectives:

1. Get your backlogs in shape!
2. Make a few more decisions that should be made before Sprint 1

Overall Requirements:

- Backlog refinement/grooming meeting: get your product backlog ready
- Sprint planning meeting: get your Sprint 1 backlog ready
- Make a few architectural decisions
- Draw up your initial data model

Tasks:

1. **Product Backlog** ► Let's get your product backlog into shape. Gather with your team for a Backlog Grooming & Refinement meeting. Here are some recommended steps:
 1. Choose a team member to be the Scrum Master for this week and the upcoming Sprint 1. Have this person lead the meeting and try to follow the principles we've learned about Scrum.
 2. Start with your epics. Epics for all core features (that you identified in your mindmap and needs & features lists) are typed into Jira and prioritized in the Epic panel. Make sure your team is in agreement concerning the ordering. Remember, you don't have to do all the stories in an epic in order; you can choose stories from multiple epics to go into one sprint. But those stories will naturally come from the top priority epics. I recommend having one epic called "Initial Application Setup" that will hold user stories that allow you to get the base application up and running.
 3. All user stories that you identified last week have been entered and prioritized. Now look at those and your top epics. Break down any epics (or user stories that are in fact of epic size) into **good** user stories following the INVEST attributes. Write descriptions for those user stories that follow the exemplar. Add preconditions, and dependencies if there are any. Estimate their effort, talk about what will be required to implement them (write a task or two), prioritize, do some modeling, and other activities you saw in the Scrum Videos and read about in the Scrum Guide. You'll have one more chance to look at these when you do Sprint Planning. Your goal here is to have enough user stories for 1 ½ sprints. (See below for how many effort points that is.)

Each user story here **must** be a complete feature representing a thin vertical slice, and meeting as much of the INVEST qualities as you can. *It's really important to try hard to get these right. User stories that don't meet these criteria are difficult to complete and you're likely to have a hard time with them and end the sprint with a lower grade.*
4. Repeat until you have a backlog you think is realistic and will get your project started.

2. **Sprint Backlog** ► Now for the Sprint backlog! Your Scrum Master should organize, and your team should then hold, a Sprint planning meeting. If your grooming and refinement meeting was a good one, this planning meeting will be short and easy. If not, you've got some more work to do. You should really have some goals for the sprint -- things you hope to achieve (and build).

Go down your product backlog one user story at a time. The Scrum Master should ask these questions at least:

- How do you feel about this user story? Does it make sense and does it help accomplish the goals of this sprint and the project?
- Is it well defined? Are there any questions about how it will be implemented?
- Is it a vertical slice?
- How big is it? Re-estimate when there is any doubt.
- Can we identify any more tasks?
- Can we answer some questions by doing some modeling? UI, database, UML diagrams?
- Can we answer some questions by looking something up?
- And lastly, can we commit to building this feature during this sprint?

Create a sprint in Jira, giving it the correct dates, etc. Sprints start the day after the meeting with your advisor and end two weeks later on the day of the meeting, i.e. the sprint review meeting is on the last day of the sprint. Add user stories to the sprint. Your team needs to have enough stories in the sprint so that **each team member has 6 effort points per sprint**. That's $6 * 4 = 24$ effort points total for a 4 person team. (Recall that effort points can only be 1, 2, or 4 for this first sprint.)

Ideally you want each team member to select their own stories on a just-in-time basis, limiting WIP (work in progress) to one story per person at a time. For our sprints, however, go around and choose the entire sprint's worth of stories for each person to equally distribute the load and the points. But you only work on one user story at a time!

Be ready for the start of the sprint after your review meeting next week. Plan to have 4 points worth of stories done per developer within the first week, and earlier if there are dependencies with a teammate's story. **Do not under any circumstances think that you have 2 weeks to do this work and then wait until late in week 2 to start or do all your work. Imagine if you were a professional developer and did that in your job. What would happen?** If you are sick or otherwise unable to work on your project, immediately alert your teammates and then your project advisor. Teammates should do what they can to continue with the project. We will deal with events like this on a case-by-case basis.

Here's a preview of what happens at the end of the sprint. At the sprint review meeting we will begin by reading a user story description. Then your team demos the completed feature on the deployed site. The client (in this case that'll be the instructor playing that role) decides if it is correct, done and acceptable. Then we'll move to the next story. **I can't say this with enough emphasis: the description and the completed work must match and it must meet all acceptance criteria!**

Where do things typically go wrong?

- A description was written but it wasn't clear and detailed and a feature built that may or may not meet the requirements
- No description was written but a feature was built anyway
- A description was written but no feature was completed
- A seemingly good description was written but it was poorly estimated, usually the work required was more difficult than anticipated. Feature completed correctly, partially or not at all.
- No acceptance criteria (or tests) were written
- No tests were written

3. **Architectural Decisions** ► There are some things your team should really decide on before starting. I've called these architectural but they're not necessarily that. They are more "structural" probably. Decide on the following, and write them down in your repository in the appropriate place:

1. What folder structure and naming convention will you use for your project source code? You'll have at least 4 projects: 1) the main project, 2) NUnit testing project, 3) BDD testing project, and 4) Jest javascript testing project. Your test projects should all begin or end in the word `Tests_` or `_Tests`.
2. What .NET Core version to use?
3. What front-end CSS library and version are you going to use? I recommend the latest version of Bootstrap, but we have had teams use other libraries. Two years ago a team went with a really nice minimalist design and used Ulkit. I've used Materialize for it's smaller and simpler approach. Tailwind CSS seems to be really popular right now. You may also want to talk about whether or not theme-ing is important for your project.
4. Decide on how you'll use Javascript: jQuery (which version) or go for plain old JS. Any other libraries you think you'll want to use? Regardless of your decision, **all** team members **must** follow the same convention -- no mix and match.
5. How will you name your Git branches? Choose a strategy. Unfortunately it looks like the "Create Branch" feature in Jira is only available for Bitbucket repositories or GitHub repos with an Enterprise Account. It won't create a branch for you with a personal account but it will recommend names.
6. How will you write your database scripts, table names, PK and FK names? *Because of some tools I'll give you later, I'm making this decision for you. See the example I'll put in the module. It'll be required that everyone follow this example to the letter!*
7. Will you use eager loading or enable lazy loading of related entities in Entity Framework Core?

4. **Initial Data Model** ► This one's too critical to wait for. As a team, decide on and actually write the DDL (up and down) scripts for your *initial* data model. Make sure you have both the source and diagram in or accessible from your repo. This is only the initial version, but it should capture all the entities and relations for all the user stories you've committed to building during sprint 1 (and **only** sprint 1). It's quite likely that you won't get it right this first time, and that's OK. It's just far better if the whole team has something to start from at the outset, and the user story that sets up the project can at least get some models constructed. Then you can edit/refine/refactor all you like during your actual user story implementation.